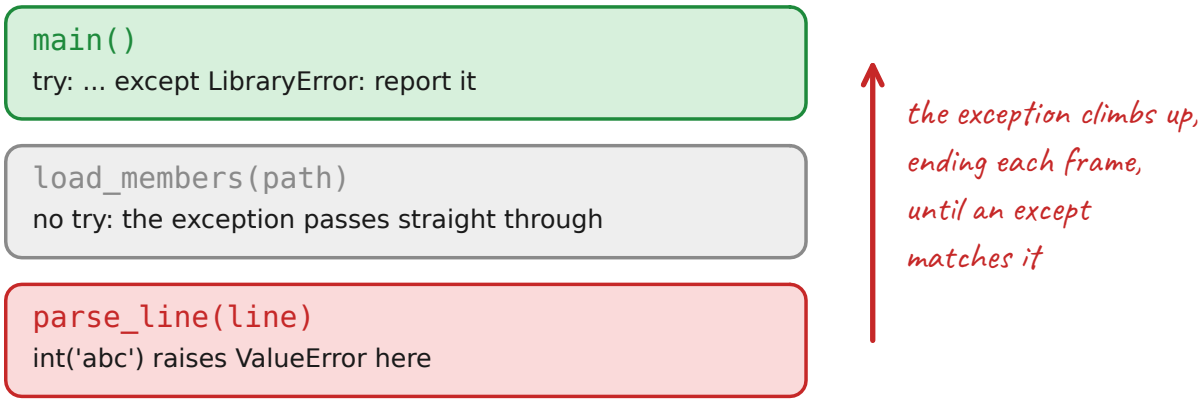
Chapter 6

# Errors and context managers

*errors climb the call stack until someone catches them*

When something goes wrong, Python **raises** an exception: an object describing the problem. The current function stops at once, and the exception travels **up** the call stack, ending each function it passes through, until it reaches a `try` block with a matching `except`. If none matches, the program stops and prints a traceback.




the call stack: each call adds a frame below its caller

```python
import traceback

def parse_line(line):
    name, age = line.split(",")
    return name, int(age)

def load_members(lines):
    return [parse_line(ln) for ln in lines]

try:
    load_members(["ana,20", "ben,abc"])
except ValueError:
    print(traceback.format_exc(limit=-2))
```

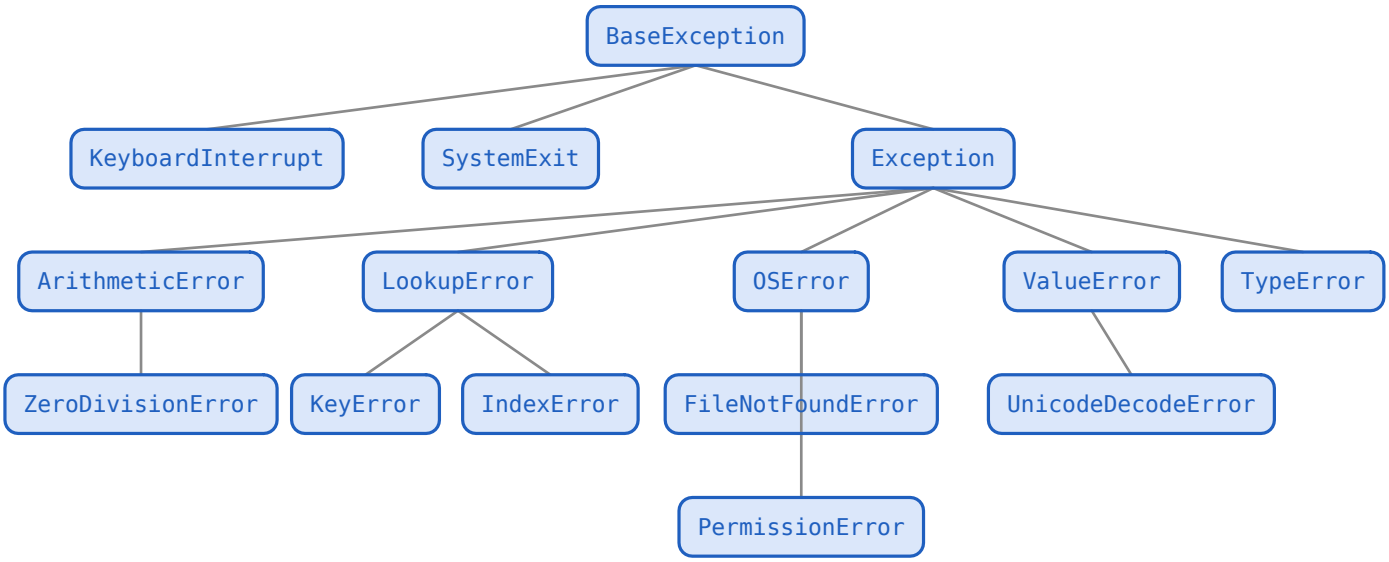
```
Traceback (most recent call last):
  File "<chapter 6>", line 8, in load_members
  File "<chapter 6>", line 5, in parse_line
ValueError: invalid literal for int() with base 10: 'abc'
```

**RULE OF THUMB**

Read a traceback from the **bottom**: the last line says what happened (`ValueError: invalid literal...`), the frame just above it says where. Then walk upwards to find the first line that's **your** code doing something wrong.

## The exception family tree

Exceptions are classes, arranged in a hierarchy. `except X` catches `X` **and all its subclasses**, so where you catch in the tree decides how much you catch:




```python
def lookup(data, key):
    try:
        return data[key]
    except LookupError as e:                # catches KeyError AND IndexError
        return f"missing: {type(e).__name__}"

print(lookup({"a": 1}, "b"), lookup([10], 5), lookup([10], 0))
```

```
missing: KeyError missing: IndexError 10
```

**TRAP**

Never write a bare `except:` or `except BaseException:`. They also catch `KeyboardInterrupt` (Ctrl+C) and `SystemExit`, so your program can't be stopped. Even `except Exception:` hides bugs like typos (`NameError`). Catch the most specific class you can actually handle.

**TRAP**

Order matters: Python checks `except` clauses top to bottom and runs the first match. `except Exception` above `except ValueError` means the `ValueError` branch can never run.

# try, except, else, finally

A `try` statement has up to four parts. Watch which ones run when the body succeeds and when it fails:

```python
def attempt(text):
    try:
        print("  try:", end=" ")
        n = int(text)
    except ValueError as e:
        print("except:", type(e).__name__, end=" | ")
    else:
        print("else: got", n, end=" | ")      # only if no exception
    finally:
        print("finally")                      # always, even on return or a new error

attempt("42")
attempt("4x2")
```

```
  try: else: got 42 | finally
  try: except: ValueError | finally
```

| part | runs when | use it for |
|---|---|---|
| `try:` | always (until something raises) | only the lines that can fail |
| `except E as e:` | an `E` (or subclass) was raised in the `try` | recovering, or translating the error |
| `else:` | the `try` finished without an exception | the next steps, kept out of the `try` |
| `finally:` | always, whatever happened | cleanup: close, release, delete temp files |

## Ask forgiveness, or look before you leap?

### LBYL: check first

```python
import os

path = "members.csv"
if os.path.exists(path):
    with open(path) as fh:
        print(fh.read())
else:
    print("no file")
```

```
no file
```

### EAFP: try it

```python
path = "members.csv"
try:
    with open(path) as fh:
        print(fh.read())
except FileNotFoundError:
    print("no file")
```

```
no file
```

no race between check and use

clear when failure is common

**WHY**
The LBYL version has a race: the file can be deleted between `exists()` and `open()`. Python style generally prefers EAFP for things outside your control (files, network, dict keys from input), and plain `if` checks for your own program's logic.

## Your own exceptions, and chaining

A library or app should raise exceptions that mean something in **its** terms. Make one base class, then specific subclasses; callers can catch broadly or narrowly:

```
class LibraryError(Exception):
    """Base class for everything this app raises on purpose."""

class LimitReached(LibraryError):
    def __init__(self, member, limit):
        super().__init__(f"{member} already has {limit} loans")
        self.member, self.limit = member, limit

class UnknownBook(LibraryError, KeyError):     # also a KeyError, for code that expects one
    pass

try:
    raise LimitReached("ana", 3)
except LibraryError as e:
    print(type(e).__name__, "-", e, "-", e.limit)
```

```
LimitReached - ana already has 3 loans - 3
```

When you catch a low-level error and raise your own, use `raise ... from err`. The original is kept as `__cause__` and printed too, so nobody loses the real reason:

```
import traceback

BOOKS = {"111": "Dune"}

def find(isbn):
    try:
        return BOOKS[isbn]
    except KeyError as err:
        raise UnknownBook(isbn) from err

try:
    find("999")
except LibraryError as e:
    print(repr(e), "caused by", repr(e.__cause__))
    print(traceback.format_exc().count("direct cause"), "'direct cause' line in the traceback")
```

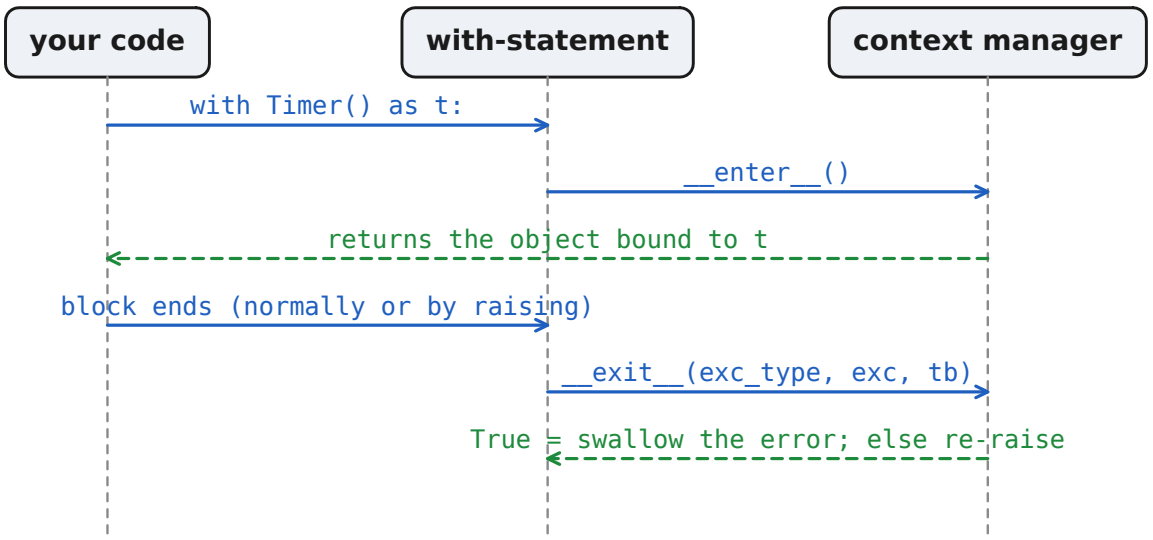
```
UnknownBook('999') caused by KeyError('999')
1 'direct cause' line in the traceback
```

**KEY IDEA**

Since 3.11 you can attach context without changing the exception: `e.add_note("while importing row 17")` adds a line to the traceback. Use it in loops over input data, where the row number is what you actually need to know.

## with: cleanup that can't be forgotten

A `with` block guarantees that setup is paired with cleanup, even when the body raises. It calls `__enter__` at the start and `__exit__` at the end, **whatever happens**:




```python
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self                             # this is what 'as t' receives

    def __exit__(self, exc_type, exc, tb):
        self.ms = (time.perf_counter() - self.start) * 1000
        print(f"block took {self.ms:.2f} ms, error: {exc_type.__name__ if exc_type else None}")
        return False                            # don't swallow exceptions

with Timer() as t:
    sum(range(100_000))
try:
    with Timer():
        1 / 0
except ZeroDivisionError:
    print("the error still reached us")
```

```
block took 0.85 ms, error: None
block took 0.00 ms, error: ZeroDivisionError
the error still reached us
```

## The quick way: a generator with @contextmanager

```
1  import contextlib, os, tempfile
2
3  @contextlib.contextmanager
4  def working_dir(path):
5      old = os.getcwd()
6      os.chdir(path)                  # setup: everything before yield
7      try:
8          yield path                  # the with-block runs here
9      finally:
10         os.chdir(old)               # cleanup: runs even if the block raised
11
12 with tempfile.TemporaryDirectory() as d, working_dir(d):
13     print(os.getcwd() == os.path.realpath(d))
14 print(os.getcwd() == os.path.realpath(d))
```

```
True
False
```

*line 8: without try/finally, an error skips cleanup*

| stdlib context manager | guarantees |
|---|---|
| `open(...)` | the file is closed (and flushed) |
| `sqlite3` connection | commit on success, rollback on error (Ch 12; doesn't close!) |
| `threading.Lock()` | the lock is released (Ch 16) |
| `tempfile.TemporaryDirectory()` | the directory and its contents are deleted |
| `contextlib.suppress(E)` | errors of type `E` are ignored, nothing else |
| `contextlib.ExitStack()` | a variable number of resources, all cleaned up in reverse order |

## Several errors at once, and logging instead of print

When many independent tasks fail together (validating every row, running tasks concurrently), you want **all** the errors, not just the first. Python 3.11 added `ExceptionGroup` and `except*`:

```
def validate(rows):
    errors = []
    for i, (name, age) in enumerate(rows, 1):
        if not name:
            errors.append(ValueError(f"row {i}: empty name"))
        if not 0 < age < 130:
            errors.append(ValueError(f"row {i}: bad age {age}"))
    if errors:
        raise ExceptionGroup("invalid rows", errors)

try:
    validate([("ana", 20), ("", 200), ("cy", 30)])
except* ValueError as group:
    for e in group.exceptions:
        print(e)
```

```
row 2: empty name
row 2: bad age 200
```

`print` is for output the user asked for. Diagnostics belong in `logging`: it has levels, timestamps, one place to configure where messages go, and it can be turned down without editing code:

```
import logging, sys

logging.basicConfig(stream=sys.stdout, level=logging.INFO, force=True,
                    format="%(levelname)-7s %(name)s: %(message)s")
log = logging.getLogger("library.loans")

log.debug("not shown: below INFO")
log.info("lent %s to %s", "Dune", "ana")    # args, not an f-string: formatted only if shown
try:
    {}["999"]
except KeyError:
    log.exception("lookup failed")                  # ERROR level + the traceback
```

```
INFO    library.loans: lent Dune to ana
ERROR   library.loans: lookup failed
Traceback (most recent call last):
  File "<chapter 6>", line 10, in <module>
KeyError: '999'
```

| level | for |
|---|---|
| `DEBUG` | details only useful while diagnosing |
| `INFO` | normal events worth a record (started, lent a book) |
| `WARNING` | unexpected but handled (retrying, using a default); the default threshold |
| `ERROR` / `CRITICAL` | something failed / the program can't continue |

**TRAP**

`assert` is for **your own** assumptions while developing ("this list can't be empty here"), never for checking user input: `python -O` removes every assert. Validate input with `if ...: raise ValueError(...)`.

# ▢ Practise, then stamp

**NOW PRACTISE (practice/ folder, no installs needed)**

Run `python -m practice check 6.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 6.1` shows other valid solutions and their trade-offs.

`6.1` fix the bug The except that hides everything
`6.2` build from a spec All or nothing: a transaction context manager
`6.3` build from a spec Temporarily change environment variables
`6.4` build from a spec Exceptions that mean something
`6.5` build from a spec Retry, and keep the real cause
`6.6` mini-project Mini-project: report every bad row at once

**PASSPORT STAMP** ask the same question of every new language (Appendix B)

**8. Errors**

Exceptions are class instances that unwind the stack until a matching `except` (subclasses match). `try/except/else/finally`; `raise X from err` chains; `ExceptionGroup` + `except*` for many at once. Style: EAFP. Cleanup via `with` (`__enter__`/`__exit__`), never by hoping.

**12. Stdlib and tooling**

`logging` for diagnostics; `contextlib` (contextmanager, suppress, ExitStack) for resources.

**IN OTHER LANGUAGES**

Java has checked exceptions (the compiler forces you to handle them) and try-with-resources for cleanup. Go has no exceptions: functions return an `error` value you must check. Rust returns `Result<T, E>` and uses `?` to pass errors up. C++ uses destructors (RAII) where Python uses `with`.

**DEEP DIVE (your vault)**

Vault notes: 6.12 Exceptions as non-local control flow · 6.13 Exception propagation and stack unwinding · 6.14 Exceptions versus result values · 6.15 Iteration, exceptions, and generator cleanup · 11.1 Boundaries turn assumptions into failure modes · 11.4 Context managers and deterministic release · 12.11 Debugging as hypothesis testing · 12.12 Logging and diagnostics · 15.9 Exception machinery.